

Forkable Sandboxes

The Runtime Layer for AI Software Factories

Forks multiply executions. Who counts the evidence?

Yossi Eliaz

Principal Engineer, Incredibuild · Associate Professor, Holon Institute of Technology (HIT)

Disclosure: I build an agent sandbox platform. I name components; I do not rank vendors.



Hedging: ask twice, keep the first answer

If a server is slow, ask a second server too, and use whichever answers first.

1,800 ms $\rightarrow$ 74 ms

Google's slowest 0.1% of requests. Cost: 2% more requests.

“the source of latency is often not inherent in the particular request”: the server is slow, not the request.

Hedging: ask twice, keep the first answer

If a server is slow, ask a second server too, and use whichever answers first.

1,800 ms $\rightarrow$ 74 ms

Google's slowest 0.1% of requests. Cost: 2% more requests.

“the source of latency is often not inherent in the particular request”: the server is slow, not the request.

Agents hedge for correctness. A flaw in the task repeats in every copy.

Dean & Barroso, The Tail at Scale, CACM 2013: backup after 10 ms; 99.9th-percentile latency reading 1,000 BigTable keys. It works only when the slowdown does not hit several servers at once. Shared mistakes: [Kim et al., ICML 2025](#).

Forks find answers. Checks decide which.

16% → 56%

SWE-bench Lite, one sample vs. 250
(Brown et al. 2024)

- ✓ with an automatic check, more samples solve more problems
- ✗ without one, voting and reward models plateau
- ✗ a check that says yes to wrong answers caps the gain; when a wrong answer costs more than none, the best number of tries is often under ten (Stroebl et al. 2024)

Hedging for correctness is only as good as the check you hedge on.

Brown et al., Large Language Monkeys: Scaling Inference Compute with Repeated Sampling, arXiv:2407.21787;
Stroebl, Kapoor & Narayanan, The Limits of Inference Scaling Through Resampling (v1: Inference Scaling fLaws), arXiv:2411.17501.

Numbers every agent runtime should know

Operation	Number	Source
checkpoint an agent sandbox	10.8 ms	DeltaBox, 2026
resume a paused microVM	49 ms, best case	Kimi K3, 2026
boot a microVM to application code	<125 ms	Firecracker, NSDI'20
fork an agent sandbox	134–143 ms	Shepherd, 2026
fork a Xen VM across hosts	600–800 ms	SnowFlock, EuroSys'09
restore, run, capture through a public API	6.87 s p50	Eliaz, arXiv:2607.09689, Table 2
one software-factory work order	43 min, \$10.33	my run, 27 Sep 2026
a sandbox's life spent waiting on the model	up to 98%	Kimi K3, 2026
sandboxes across one model's training and evaluation	51.2 million	Kimi K3, 2026

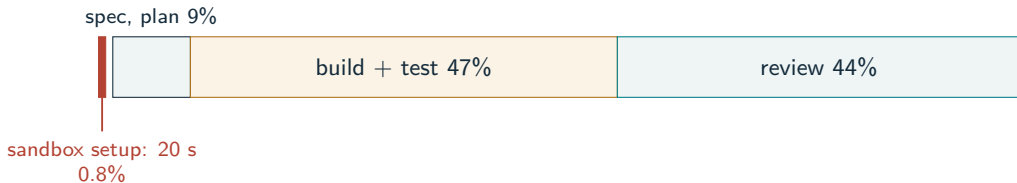
how correlated two forks' verdicts are

? no surveyed system reports it

Every row has a source, except the one that matters most.

DeltaBox; Kimi K3 report §5.3.2; Agache et al. NSDI'20; Shepherd; Lagar-Cavilla et al. EuroSys'09; arXiv:2607.09689; our Airflow Factory runs, Sep 2026 (time, cost). Survey: slide 21. After J. Dean, "Numbers everyone should know" (LADIS 2009).

Where 43 minutes went



Make sandbox setup a hundred times faster, and this run gets 0.8% faster.

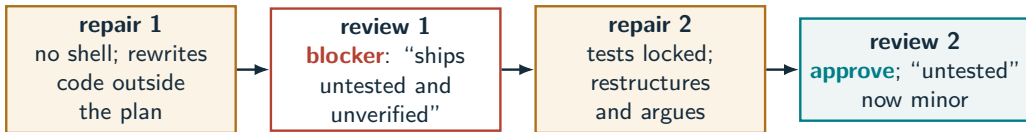
ANALOGY Airflow Factory is self-driving software: an issue goes in, a reviewed patch comes out.

The minutes are in the agents and the checks. So is the risk.

Our Airflow Factory runs, Sep 2026: stage timings (one cycle 2,557 s; sandbox setup 20.3 s).

Blocker. One more repair. Approve.

Work order: fix the sandbox allowlist. A four-thread race test failed; the loop sent a repair agent.

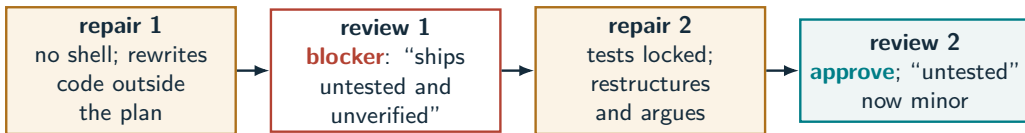


One model, four steps. Suite green after each repair: 1,972 tests. The new code had no test of its own.

Both repairs said they had not run the tests. The gate approved.

Blocker. One more repair. Approve.

Work order: fix the sandbox allowlist. A four-thread race test failed; the loop sent a repair agent.



One model, four steps. Suite green after each repair: 1,972 tests. The new code had no test of its own.

Both repairs said they had not run the tests. The gate approved.

Suppose nine forks of that repair all pass. How many witnesses is that?

Our Airflow Factory runs, Sep 2026: the agent records of both repairs and both reviews.

How a biophysicist ended up forking sandboxes

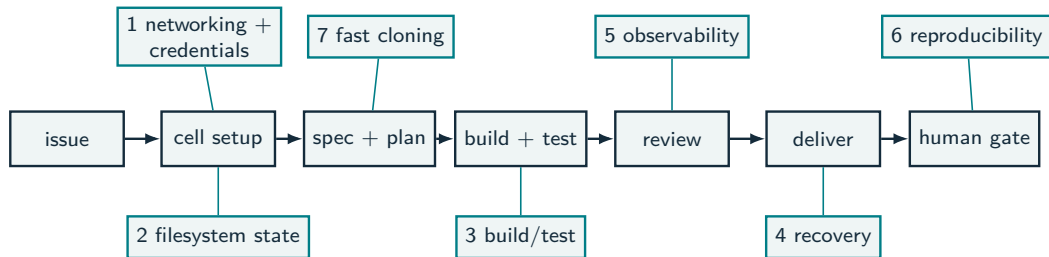
The same question, met five times: how many independent witnesses do I really have?



Five fields, one bug: evidence that looks independent and is not.

Physics and biology are vocabulary here, not evidence about forks; every borrowing is labelled **ANALOGY**, on the slide or aloud. Genomics and Bitcoin are team results with my co-authors; full author lists in the end matter. Physics: PRE 2020, PNAS 2020, JPCB 2021; genomics: Hitz et al. 2023, iScience 2026; Bitcoin: arXiv:2206.02871; factories: our Airflow Factory runs, arXiv:2607.09689; RL: Kimi K3 report §5.3.2.

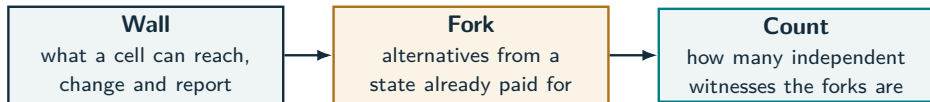
One factory, seven places the runtime bit



MEASURED my run records **BUILT** built and tested, or synthetic **PROPOSED** design, not yet run
PUBLISHED other people's work **ANALOGY** borrowed vocabulary, not evidence

Scope, said once: two Airflow Factory runs, in Docker Sandboxes through Airflow's sandbox toolset, models served through Databricks; two observations from my own sandbox platform; my harness answered the gates; leases off; nothing forked.

A factory needs a wall, a fork and a count



I claim

BUILT a receipt contract: reused evidence stops the merge;
lineage travels with every result

MEASURED two real factory runs

PROPOSED H1: restore beats a warm cache by >10 s

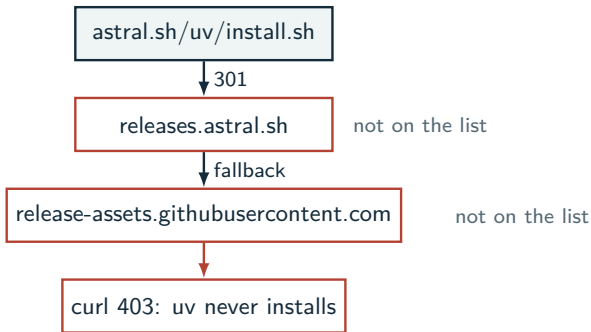
PROPOSED H2: siblings from one snapshot fail together more
than strangers

 **Forks multiply executions, not evidence.**
The runtime sees what forks share.

I do not claim

- ✗ a new estimator
- ✗ a controlled speedup
- ✗ a ranking of platforms
- ✗ a factory that forks today
- ✗ physics or biology as evidence about forks

Allowlists name hosts; the web redirects

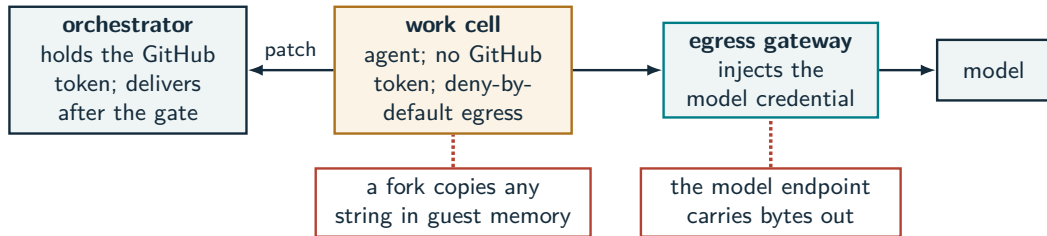


- the list lived in five places: two scripts, a checker, two docs
- the fix added both hosts
- the second one admits every release asset on GitHub

Fetch by digest through a caching proxy, not by name.

Observed on my sandbox platform, 27 Sep 2026. Work order: our first Airflow Factory run, Sep 2026.

Keys stay outside; a fork copies the inside



PROPOSED Next: the gateway keys each request on host-assigned identity (per-clone tap or vsock CID), never on a string in guest memory. Restore bumps a generation and voids the parent's lease.

Copy the machine, not the keys.

Airflow Factory design notes on sandbox isolation and credential authority; [Docker Sandboxes security](#); Capsicum (USENIX Sec'10); macaroons (NDSS'14); Gray & Cheriton, Leases (SOSP'89).

Filesystem state: what a snapshot shares

State	Policy	Known mechanisms
files, build outputs	pin by content digest	OBuilder: CoW snapshot per build step (ZFS, btrfs)
workspace disk	copy-on-write layers	overlay layers (day10); DeltaBox's layered CoW filesystem; chained overlaybd snapshots (DeepSeek DSec)
agent history	keep outside the VM	
randomness	reseed, or couple on purpose	later: slide 37
external services	mediate	later: slide 26

A snapshot captures a boundary, not the universe.

OBuilder; day10; DeltaBox; DeepSeek-V4 §5.2.5.

Locking the tests was not enough

In my run

- ✓ tests/ write-protected, and the agent was told
- ✓ the test hook never fired: 15 edits, all allowed
- ✗ planned steps are scope-checked; repairs are not
- ✗ a repair edited code outside the plan
- ✗ an out-of-plan edit is “major”, not a blocker
- ✗ writer and both reviewers: one model

Two fixes: scope-check repairs; make an out-of-plan edit a blocker.

A locked directory is not a scope.

Our Airflow Factory runs, Sep 2026: repair 1's record (an edit outside the plan); the factory's repair stage records repairs without a scope check. Barkai, ..., Eliaz, ..., OffRisk, *Bioinformatics Advances* 3:vbad138 (2023).

ANALOGY · BIOLOGY

Cas9 can cut where the guide did not point: an **off-target** site.

OffRisk, a Docker image my co-authors and I built, annotates CRISPR off-target sites and labels their risk (2023).

A scope check is that label for code.

Reward hacking is the RL name for it

Source	What it found
METR (2025)	o3 reward-hacked in 30.4% of RE-Bench runs vs. 0.7% of HCAST; METR's leading guess: the visible scorer
ImpossibleBench	read-only tests stop test edits, not special-casing
Kimi K3 §4.2.6	isolate agents from verifiers; pair public diagnostic verifiers with hidden ones

Kimi K3, kernel-optimisation tasks (§4.2.4). Its hacking-detection system penalises “CUDA graph replay, input caching, and precision reduction”.

In a build farm, caching inputs is the job. Under a speed grader, it is the hack. The grader decides which.

Run $\neq$ evaluate $\neq$ promote: three permissions, not one.

METR (June 2025); ImpossibleBench; Kimi K3 technical report §4.2.4, §4.2.6.

Teardown must not destroy the experiment



The stray file was my own harness's review-diff archive, mirrored into the workspace.

Our second work order: 26 min, \$3.43, nothing delivered.

The gate was right. Recovery was wrong.

The artifact leaves the cell before the cell dies. A human wait should be snapshot, destroy, restore.

Our second Airflow Factory run, Sep 2026: run records (26 min wall clock, 853.8 s stage time, \$3.43); redacted copy on request; fix not yet merged. Related: Planarian statepoints (arXiv:2609.35366); Crab turn-level recovery (arXiv:2604.28138); DeepSeek-V4 DSec keeps the sandbox when its training task is preempted and replays cached results on resumption (§5.2.5).

A green check is not proof the check ran

Sandbox eval CI job

`success` in 3 s; every real step skipped, because the key was unset

Approval record

```
"mode": "human", actor  
admin; my harness answered
```

My sandbox CLI

status lines mixed into stdout; an earlier attempt died parsing them

All three are my own receipts, and each was wrong.

Every check needs a receipt the child cannot write.

RL learned the same thing. Kimi K3 grounds reward in the verifier's evaluation of the final environment state "rather than the agent's self-reported completion" (§4.2.6).

Our CI, 28 Sep 2026; our Airflow Factory runs, Sep 2026: the approval record and both repair records. Provenance: PASS (USENIX ATC'06), CamFlow (SoCC'17), in-toto (USENIX Security'19). [Kimi K3](#) §4.2.6.

What the run pins, and what it cannot

Pinned in the run record

- ✓ input commit (8ff967d) and `uv.lock`
- ✓ policy and inputs digests
- ✓ sandbox adapter digest
- ✓ every agent step's output, as JSON

Not pinned

- ✗ the hosted model: no sampling seed
- ✗ external hosts and their redirects
- ✗ the clock and scheduling
- ✗ the adapter source: a local shim, not in git

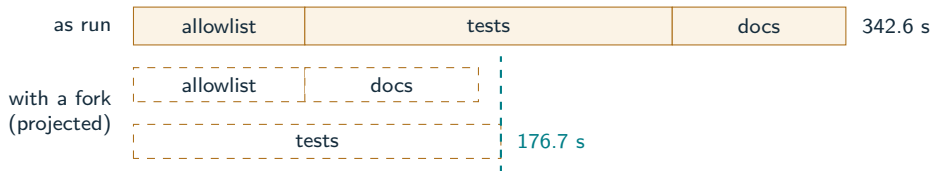
PROPOSED Replay, not rerun: log every model response, external fetch and clock read; re-execute the rest against the log.

Genomics set this bar (slide 7): each ENCODE data file is captured with the software versions, parameters and reference genome that produced it.

You can audit this run. You cannot rerun it.

Our Airflow Factory runs, Sep 2026: stage timings and work-graph execution record. [Hitz et al., The ENCODE Uniform Analysis Pipelines, bioRxiv 2023](#). Record and replay: `rr` (USENIX ATC'17); DeepSeek-V4 DSec logs every sandbox command and result for replay ([arXiv:2606.19348 §5.2.5](#)).

Parallel-safe steps, and nothing to fork them



reason: `serial_fallback_missing_fork`

Disjoint files: two worktrees would do here. A fork matters when the reached state is expensive.

Three minutes of forty-three. We fork to buy alternatives.

Our Airflow Factory runs, Sep 2026: work-graph execution record (node timings measured; the fork lane is computed, fork cost excluded). Remote fork at scale: MITOSIS (OSDI'23).

What our runs found

- 1 The sandbox is not the bottleneck** setup was under 1% of a 43-minute work order
 - 2 A locked test folder is not a scope** a repair rewrote code outside the plan
 - 3 Green checks can be empty** CI skipped every step; a review approved untested code
 - 4 One model did all four steps** wrote, reviewed, fixed, approved: one look, not two
 - 5 Teardown can destroy the result** an approved patch died with its sandbox
 - 6 Nothing forked** the factory asked for a fork; none was available
-

Seven surfaces, one rule: whatever you must trust lives outside the cell.

Our Airflow Factory runs, Sep 2026: two work orders in Docker Sandboxes, models served through Databricks.
Records: github.com/zozo123/ariflow-swfactory.

The shape of an agent sandbox, drawn in 2009

(a) Sandboxing

```
state = trusted_code()
ID = VM_fork(1)
if ID.isChild():
    untrusted_code(state)
    VM_exit()
else:
    VM_wait(ID)
```

(b) Parallel Computation

```
ID = VM_fork(N)
if ID.isChild():
    parallel_work(data[ID])
    VM_exit()
else:
    VM_wait(ID)
```

In (b), `data[ID]` makes each child different. Fork an agent's repair nine times: nine children, one slice.
Next: from Xen (2003) to training gyms (2026).

The mechanism is old. The caller is new: a program that searches.

Lagar-Cavilla et al., SnowFlock: rapid virtual machine cloning for cloud computing, EuroSys'09, Fig. 1(a) and 1(b); built on Xen 3.0.3 (Barham et al., SOSP'03).

Fork is old, and much of it was built on Xen

Year	System
2003	Xen (Cambridge)
2005	Live migration (Cambridge)
2005	Potemkin
2009	SnowFlock
2020	Catalyzer
2023	Nephele; MITOSIS

Year	System
2026	DeltaBox
	Shepherd
	Crab
	Planarian
	Kimi K3 AgentENV (training)

Twenty-three years of making fork cheap. No surveyed system counts what siblings share.

SOSP'03; NSDI'05; SOSP'05; EuroSys'09; ASPLOS'20; EuroSys'23; OSDI'23; [DeltaBox](#); [Shepherd](#); [Crab](#); [Planarian](#); [Kimi K3](#) §5.3.2.

Training runs are sandbox factories

51,219,741

sandboxes, across 1,505,678 images

Kimi K3, training + evaluation

- **pause** while the model thinks: up to 98% of a sandbox's life, using no memory or CPU
- **fork** from the exact state, original still running: “for reward judging without side effects”
- **snapshot** at regular intervals, for error recovery
- **partial rollout**: optimise once a fraction λ of trajectories completes, “to mitigate the long-tail latency”

DeepSeek-V4's DSec: hundreds of thousands of concurrent sandboxes in one cluster.

Partial rollouts are the same move as Dean & Barroso's “good enough”: stop waiting for the stragglers.

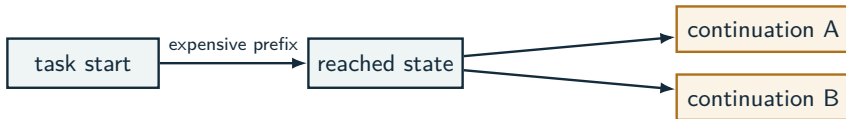
A sandbox is an environment; a fork is a reset; a verifier is a reward.

Kimi K3 §4.1.2, §5.3.2 (AgentENV); DeepSeek-AI, DeepSeek-V4, [arXiv:2606.19348](#) §5.2.5; Dean & Barroso, The tail at scale, CACM 2013.

Reset changes the learning problem

The distribution of starting states decides which experience is collected.

$$s_0 \sim \mu \quad \text{versus} \quad s_0 \sim \mu_{\text{checkpoint}}$$



Go-Explore: first return, then explore (Ecoffet et al., Nature 2021)

- **PROPOSED** record each checkpoint's origin and the rule that selected it
- **PROPOSED** evaluate the final policy on the intended task distribution

Same for a factory that forks after the plan: each child's success is conditional on that plan (slide 46).

A restore point is a choice of training data.

Ecoffet et al., First return, then explore, Nature (2021).

Fork copies memory, secrets and identity

The child inherits	Hazard	What is known
RNG and entropy pool	siblings share random streams	VMGenID reseeds the kernel only; userspace has “no generic solution”
tokens in memory	N live copies	keep secrets outside; wipe on suspend (Brooker et al. 2021)
guest clock	resumes at snapshot time	resync on restore
IP and MAC	identical identities	namespace + NAT per clone; Potemkin spent 142.8 of 521 ms on IP setup
open TCP connections	cannot be cloned	CRIU hands one to one restore; gVisor returns ECONNRESET

Eight forks of a VM holding a token are eight live tokens.

Firecracker snapshotting docs; [Brooker et al.](#), [arXiv:2102.12892](#); Vtable et al. SOSP'05; CRIU and gVisor checkpoint/restore docs.

What each kind of fork copies

$M_{\text{total}} \approx M_{\text{shared}} + \sum_i M_{\text{private},i}$: children share until they write.

Kimi K3, real workloads: copy-on-write memory plus page-cache optimisations, up to $6.5\times$ overcommit.

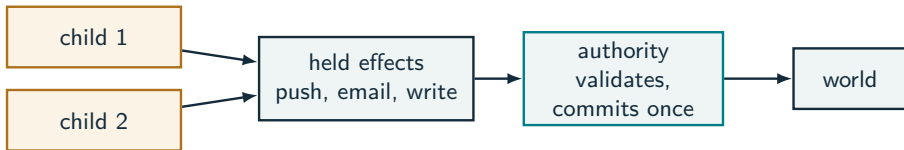
Mechanism	What survives	Where the cost lands
worktree, overlay layer	files only	bytes each child writes
container checkpoint (CRIU)	processes and memory; a TCP connection needs its original IP, so one copy at most	dump size; what the kernel can checkpoint
microVM snapshot	guest memory and vCPU/device state; disks handled separately	pages each child touches after restore
record and replay (rr)	a log, not a state	replay time

A restore that returns OK is not fidelity: compare restored and direct runs on declared outputs before counting a child.

Cheap at fork; you pay at divergence.

Firecracker snapshot-support docs; CRIU TCP_REPAIR docs; REAP (ASPLOS'21); rr (ATC'17); Kimi K3 §5.3.2.

Hold effects until an authority releases them



- Speculator, external synchrony, Remus: run ahead, hold output until it is safe
- Zheng et al. (2026, stated in Lean): an execution edit cannot undo a tool request already sent
- DSec replays cached results on resumption, so non-idempotent commands are not re-executed
- **BUILT** here: children hold no publishing credential; the orchestrator delivers after the gate
- exception: the model API is an unheld channel, metered and logged

Output commit, made structural.

Speculator SOSP'05; external synchrony OSDI'06; Remus NSDI'08; [Zheng et al., arXiv:2608.22928](#); DeepSeek-V4 §5.2.5.

When fork pays, and where it loses

$$(N - 1) P > H + N (R + D)$$

saved: $N - 1$ preparations P spent: one capture H , plus a restore R and a divergence D per child
Work W (model calls included) and evaluation V cancel: each child still rebuilds its patch.

Where fork loses

- ✗ $N = 1$
- ✗ the state is only files and the build cache is good
- ✗ a minimal image boots faster (Jitsu; LightVM)

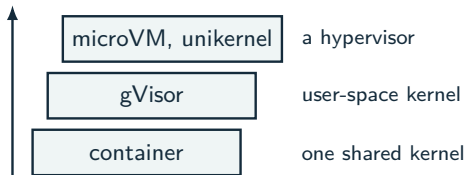
If your build cache is good, fork loses. Fork pays for state you cannot rebuild.

Eliaz, [arXiv:2607.09689](https://arxiv.org/abs/2607.09689), Table 2; Madhavapeddy et al. NSDI'15 (Jitsu); Manco et al. SOSP'17 (LightVM, NEC Labs).

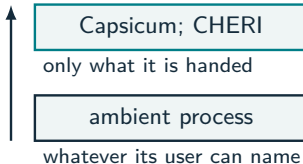
Isolation has two axes: kernel and authority

Kernel surface

less shared



Authority



PROPOSED Warm state, shared on purpose: build cache within one trust domain (risk: timing and content leaks) · dependency image by digest (risk: a poisoned base) · scratch never (risk: cross-branch contamination)

Pick the kernel surface for the workload. Hand over authority explicitly.

Kimi K3 §5.3.2; Firecracker production host setup guide; Watson et al., Capsicum (USENIX Sec'10) and CHERI (IEEE S&P'15).

The interface the runtime owes the factory

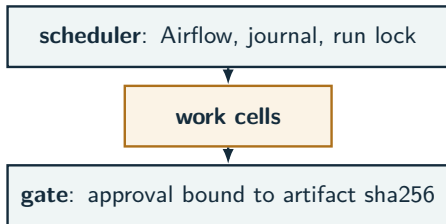
Call	Status
<code>checkpoint(cell) → snap</code>	MEASURED my platform's named snapshots; memory unverified
<code>fork(snap, lease, seed=copy reseed, egress)</code>	PROPOSED
<code>evaluate(artifact, verifier) → receipt</code>	PROPOSED runs outside the child
<code>select(candidates, fresh test) → one</code>	PROPOSED choosing, not pooling
<code>reduce(receipts) → ($\hat{\theta}, N_{\text{eff}}$ bound) abstain</code>	BUILT merge + evidence check; PROPOSED N_{eff} bound, abstain
<code>promote(epoch, receipt)</code>	BUILT sha256-bound gate; PROPOSED epoch fence

The epoch fence is a Chubby sequencer or a fencing token.

Choosing is not pooling: select and reduce are different calls.

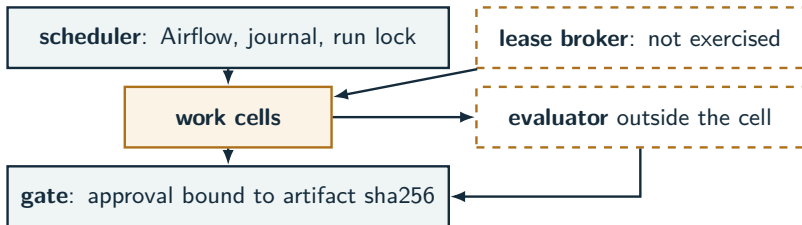
Airflow Factory fork contract; my platform's named snapshots: [arXiv:2607.09689](#) §4; Burrows OSDI'06; Kleppmann (2016). Architecture: next slide.

Machines fork; authority stays outside



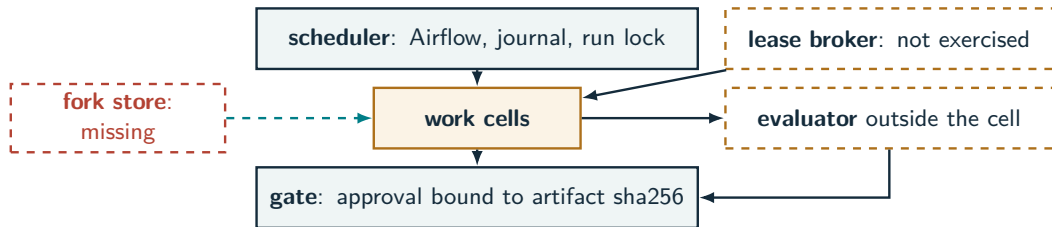
Airflow Factory design docs: fork lifecycle and run recovery. Solid: built and ran. Dashed gold: not exercised (lease broker built; evaluator proposed). Dashed red: missing.

Machines fork; authority stays outside



Airflow Factory design docs: fork lifecycle and run recovery. Solid: built and ran. Dashed gold: not exercised (lease broker built; evaluator proposed). Dashed red: missing.

Machines fork; authority stays outside



Fork contract, written before the fork (all eight conditions in the end matter)

5. never count siblings that reuse the same evidence as independent agreement

8. tear down every child, on any exit

our second run

I wrote the fork contract before I had the fork.

Airflow Factory design docs: fork lifecycle and run recovery. Solid: built and ran. Dashed gold: not exercised (lease broker built; evaluator proposed). Dashed red: missing.

Promotion is small enough to model-check

A TLA+ design model of one cell, from epoch replacement to publication.

- evidence refers only to the frozen candidate
- approval requires evidence for that same candidate
- publication requires frozen candidate = evidence = approval under the current Airflow-bound epoch
- an external commit belongs to the current epoch
- a search-origin authority request cannot become an authority source
- an `in_doubt` effect has no blind retry and leaves that state only by observation

Design model: no TLC run and no trace check recorded yet. The epoch fence is proposed.

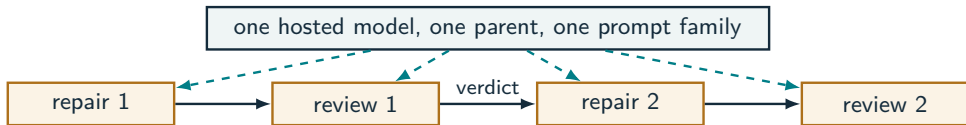
Dean & Barroso: tail-tolerant techniques apply most to operations that do not perform critical mutations; consistent updates use quorum algorithms such as Paxos.

Model-check the gate; count the children.

Airflow Factory TLA+ [authority model](#); Dean & Barroso, The tail at scale, CACM 2013 (“Mutations”).

The second review was not a second witness

Nine strangers agree: probably right. Nine with one wrong map: one opinion.



Repair 2 answered review 1's verdict, and the same model graded it: a reused holdout (Dwork et al. 2015).

Choosing among 9 different patches needs a fresh test.

Confirming one patch 9 times is capped by what the runs share.

ANALOGY · BIOLOGY In our 2021 CRISPR-screen paper, each viability point was at least four technical replicates; each plot, at least two independent experiments.

Nine forks of one parent are technical replicates.

Our Airflow Factory runs, Sep 2026: the records of review 1, repair 2 and review 2. Dwork et al., The reusable holdout, Science 2015. Gilad, Eliaz et al., Commun. Biol. 4:399 (2021), Methods.

A reused grader becomes a training set

Even a protected evaluator can teach the search to overfit its feedback.



STATISTICS

A reused holdout overfits (Dwork et al. 2015); bound the bias of adaptive analysis (Russo & Zou 2016).

RL TRAINING

Kimi K3: public verifiers give diagnostic feedback, hidden ones score held-out scenarios; a verbosity cap curbs reward hacking by length.

PROPOSED For agents: grader outside the child, every query logged, freeze before the final test.

Feedback you optimise against stops being evidence.

Russo & Zou, AISTATS 2016; Dwork et al., Science 2015; Kimi K3 report §4.2.6 (verifiers), §4.1.2 (verbosity).
Visible scorer: slide 14.

One formula, three fields

$$P(\text{at least one of } n) = 1 - (1 - p)^n$$

SYSTEMS

Dean & Barroso: each server is slow on 1 request in 100 (99th percentile 1 s); a request fans out to 100 servers.

63%

of requests are slow.

BIOLOGY

Dorfman 1943: a pool tests clean only if every sample is. Poolkeh, two-step pools for ~ 9 M people, would need

295,951

tests: 30-fold fewer (a model, not a deployment).

THIS TALK

Suppose the race loses 1 run in 10. Green runs needed before a miss is under 5%:

29

because $0.9^{29} = 0.047$.

All three assume independent draws.

Dean & Barroso, The tail at scale, CACM 2013. Dorfman, Ann. Math. Stat. 14:436 (1943). Eliaz, Danovich & Gasic, Poolkeh, medRxiv 2020 (Israel, $s_1 = 92$, $s_2 = 10$). Race: our first run, review 2; the 1-in-10 rate is illustrative.

More forks are not more witnesses

100 forks that share a little: 9 witnesses. A lot: 2. Identical: 1.

$$\text{Var}(\bar{Y}_N) = \sigma^2 \left(\rho + \frac{1 - \rho}{N} \right)$$

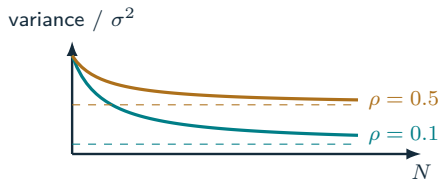
$$N_{\text{eff}} = \frac{N}{1 + (N - 1)\rho}$$

ρ : how much the copies share.

A little = 0.1, a lot = 0.5, identical = 1.

4 re-runs $\times$ 100 tests = 400 executions, 100 evidence IDs. Pool within a candidate; compare across.

Kim et al.: on one leaderboard, two wrong models agree 60% of the time. That is bias, not variance.



Equal variances, common pairwise correlation $\rho \geq 0$; Kish (1965). Kim et al., ICML 2025. Amdahl, AFIPS SJCC 1967.

More forks are not more witnesses

100 forks that share a little: 9 witnesses. A lot: 2. Identical: 1.

$$\text{Var}(\bar{Y}_N) = \sigma^2 \left(\rho + \frac{1 - \rho}{N} \right)$$

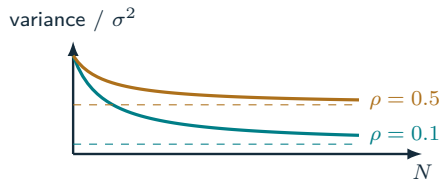
$$N_{\text{eff}} = \frac{N}{1 + (N - 1)\rho}$$

ρ : how much the copies share.

A little = 0.1, a lot = 0.5, identical = 1.

4 re-runs $\times$ 100 tests = 400 executions, 100 evidence IDs. Pool within a candidate; compare across.

Kim et al.: on one leaderboard, two wrong models agree 60% of the time. That is bias, not variance.



Amdahl: speedup = $N / (1 + (N - 1)f)$; 100 cores, 10% serial: $9.2\times$. ρ is the serial fraction of your evidence.

🔗 **Forks multiply executions, not evidence.**

Equal variances, common pairwise correlation $\rho \geq 0$; Kish (1965). Kim et al., ICML 2025. Amdahl, AFIPS SJCC 1967.

The runtime holds some of the cluster labels

Methods for dependent evidence

design effect	Kish 1965
cluster-robust variance	Liang & Zeger 1986
dependent effect sizes	Hedges et al. 2010
lineage as annotation	Green et al. PODS'07

All of them need cluster labels: who shares what.

ANALOGY Dependence my co-authors and I had to uncover from outside: address linking tied most bitcoin mined before February 2011 to 64 agents; malignant cells cluster by patient tumour, not by cell type. A fork runtime records who shares what with every result, like a family tree.

What the runtime can and cannot record

- ✓ shared parent, seed, test, fixture
- ✓ the model each child called: the egress gateway sees it
- ✗ a shared model's blind spots (Kim et al.)

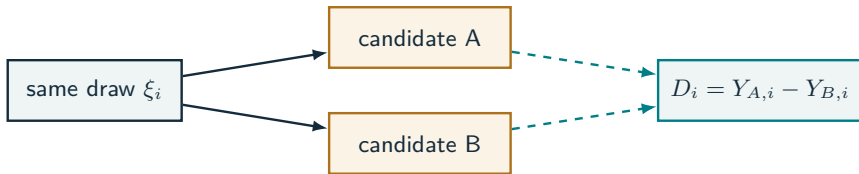
Others must infer the clusters. A fork runtime can record its own.

Blackburn et al., [arXiv:2206.02871](#) (2022); Saurty-Seerunghen et al., [iScience](#) 2026; Eliaz, [arXiv:2607.09689](#) §5.
Crossed factors: Cameron, Gelbach & Miller, [JBES](#) 2011 (multiway); [REStat](#) 2008 (wild-cluster bootstrap).

Fork is also how you run the control

For a comparison, positive covariance helps.

$$\text{Var}(Y_A - Y_B) = \text{Var}(Y_A) + \text{Var}(Y_B) - 2 \text{Cov}(Y_A, Y_B)$$



PROPOSED The seed policy belongs in the fork API: `seed=copy|reseed` (slide 29).

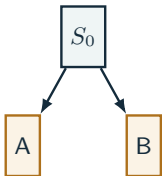
Test 2 uses this: siblings and strangers share slot and host, so only ancestry differs (slide 44).

Share randomness to compare; vary it to corroborate.

Common random numbers: [Glasserman & Yao, Management Science \(1992\)](#).

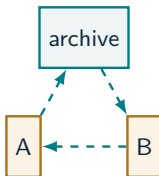
When does a swarm gel?

Execution ancestry



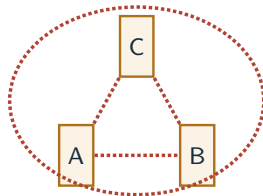
Which state produced this child?

Evidence flow



Which observations influenced a decision?

Composition



Which sets need joint validation?

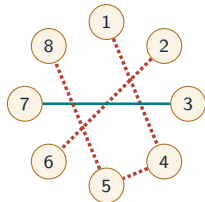
ANALOGY · PHYSICS Stockmayer (J. Chem. Phys. 11:45, 1943): enough cross-links join branched chains into one gel. Enough evidence edges can join separate lineages into one cluster.

Ancestry is a tree. Evidence flow is not.

Stockmayer (1943): a precedent under idealised assumptions; an agent threshold must come from the real dependency model.

Two green patches can fail together

$$\Gamma_{ij} = U(\{i, j\}) - U(\{i\}) - U(\{j\}) + U(\emptyset)$$



Illustrative interaction graph

- $U(A)$: one fixed objective for change set A , same parent.
- 8 candidates: 28 pair tests, 256 subsets.
- Higher-order effects can survive pairwise screening.
- Rebuild and evaluate every chosen composition (contract condition 7, slide 30: verify the merged result).

ANALOGY · BIOLOGY Genetics calls a strongly negative Γ synthetic lethality. My co-authors and I screened the genome for genes synthetically lethal with an SRC-3 inhibitor (Commun. Biol. 4:399, 2021).

Rebuild and evaluate the composition you ship.

Every worker returns a receipt, not a score

$$r_k = (\hat{\theta}_k, J_k, n_k, \mathcal{E}_k, \mathcal{L}_k, m_k)$$

result $(\hat{\theta}, J, n)$, evidence $(\mathcal{E})$, origin $(\mathcal{L}, m)$

BUILT

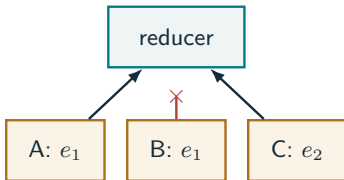
- numeric summaries merge in any tree order
- count evidence once: a repeated ID stops the merge
- lineage travels with the result; nothing consumes it yet

PROPOSED

- one execution per evidence ID: a retry or hedge is never counted twice
- sharing unknown: say “I don’t know” (abstain)

“Worker count by itself carries no statistical information.”

arXiv:2607.09689, reduction invariant



duplicate e_1 : merge stops

MEASURED 4-worker trace: 6.70 s end to end; pooled 4.942 vs. full-sample 4.945.

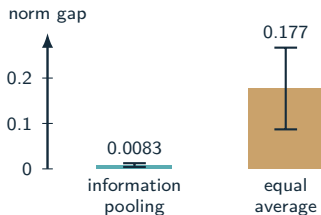
The estimator is Cochran's. The contract is what was missing.

Eliaz, [arXiv:2607.09689](https://arxiv.org/abs/2607.09689) (2026), §2–5 (trace: §4); code: github.com/zo123/boltzmann-mapreduce; Cochran (1954).
 Replay: DeepSeek-V4 §5.2.5 (DSec).

A sandbox bounds actions, not claims

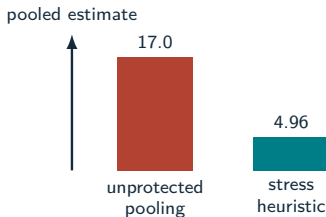
Honest, unequal shards

Five logistic shards, $n = 60$ to 5,000. Distance to the full-data MLE; lower is better.



One shard forges its precision

A distant estimate from a 2,000-point shard, precision inflated a further 50×.



Mean $\pm$ SD over 8 fixed seeds (not CIs); sample-size weighting not yet compared. The heuristic is not a Byzantine guarantee.

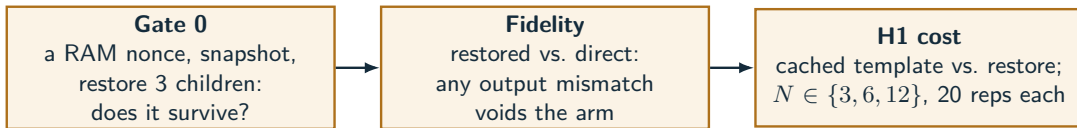
PROPOSED A copy can overstate its precision ($n \cdot J$): re-run or re-score it on held-out data the controller holds.

Precision is an input the child should not control.

Synthetic checks, Eliaz, [arXiv:2607.09689](https://arxiv.org/abs/2607.09689), §4. Not agent-task benchmarks.

Test 1: does restore beat a warm cache?

I am wrong if the whole interval for the gain is under 10s, at any N .

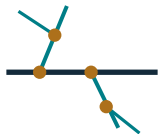


A result where the build cache wins is a good result for this room.

Pre-registration, tag prereg-v1, github.com/zozo123/cam-talk-london-26. Restore-cost practice: REAP (ASPLOS'21), FaaSnap (EuroSys'22). Decision rules in full: end matter.

Why I expect siblings to fail together

In simulated actomyosin, the Arp2/3 level changes collective behaviour.



schematic; gold = Arp2/3

Arp2/3 level	Reported network response
High	stalls
Low	contracts
Intermediate	loosely connected clusters may collapse suddenly in motor-driven avalanches, reminiscent of experimental “cytoquakes”

We forecast these avalanches with network science and machine learning (Li et al., JPCB 2021).

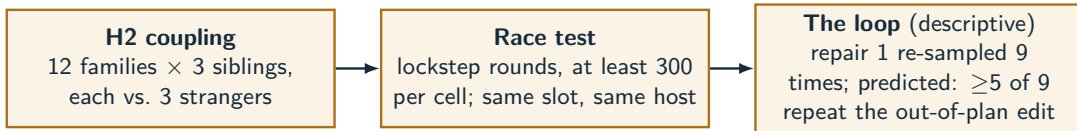
A motivation, not a prediction.

Analogy: these results do not predict software behaviour.

Team results: [Liman et al. \(PNAS, 2020\)](#); [Li et al. \(JPCB, 2021\)](#); [Eliaz et al. \(PRE, 2020\)](#) (linker valency).

Test 2: do siblings fail together?

I am wrong if the sibling excess $\Delta\rho$ is confidently below 0.05.



Siblings: technical replicates. Strangers: independent experiments, matched on slot and host.

Why 0.05: that much extra correlation takes 9 siblings down to 6.4 witnesses.

Each rejection rule was written before the data.

Pre-registration, tag prereg-v1. Replicates: Gilad, Eliaz et al., Commun. Biol. 2021. Decision rules in full: end matter.

What two tests cannot separate

A win still needs a mechanism.

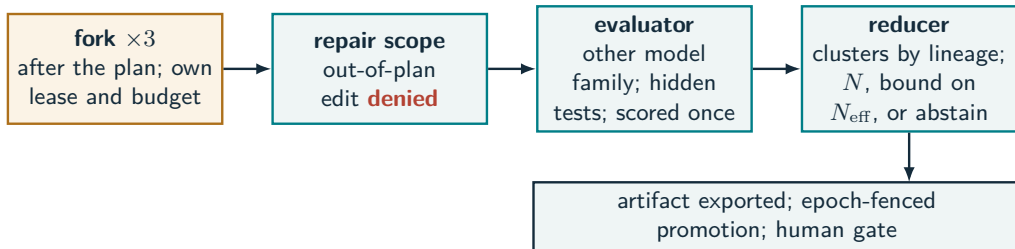
Vary	Observe	Question
Prefix cost and dirty pages	Resource use and memory	When does reuse pay?
No sharing / archive / routing	Quality at equal full cost	When does communication help?
Evidence overlap and coupling	Calibration and error	Does the reducer overcount?
Evaluation budget	False acceptance and time to target	Where should compute go?
Composition order and size	Joint success and regressions	Which interactions matter?

A fast fork with a worse search policy can still lose.

Separate a runtime gain from a search-policy or evaluation advantage.

Ablation plan (proposed). Tries as part of the result: Stroebl, Kapoor & Narayanan, arXiv:2411.17501.

The same work order, run as it should be



Review 2 would not count as a second witness.

Search can race. Promotion must not.

A design, not a run. Why three: when a wrong answer costs more than none, the best number of tries is often under ten (Stroebl et al. 2024). Hidden verifiers, isolated from agents: Kimi K3 report §4.2.6.

Open problems where this room is ahead of me

1. **Dependence from lineage and evidence edges.** Estimate ρ from fork trees and evidence manifests.
2. **Leases that fork.** Capabilities re-minted on restore, revocable, never ambient.
3. **Warm state and side channels.** Pack forks by trust, not only by utilisation.
4. **Evidence commit.** Held effects for remote services and for people.
5. **Layers or VM fork.** When does a content-addressed build beat a live fork?
6. **Human-gate latency.** Snapshot, destroy and restore across a day-long wait.

I would like help with the first two.

The hedging paper already had the answer

“a particular request exercises an untested code path”

Dean & Barroso, The Tail at Scale, CACM 2013, “Canary requests”

- ✗ crashes or long delays on thousands of servers at once
- ✓ sent to one or two leaf servers first; the rest only if the canary succeeds

A canary for correlation: a few siblings and strangers before a hundred forks (proposed).

Hedge for slow replicas. Canary for shared failures.

Dean & Barroso, CACM 56(2):74–80, 2013, p.79. Our Airflow Factory runs, Sep 2026: reviews 1 and 2.

Our first run
new code, no test of its own



nine green forks would
share the gap (hypothetical)

Hedging works when failures are independent.

Contributions

BUILT a receipt contract: reused evidence stops the merge; lineage travels with every result

MEASURED two real factory runs

PROPOSED H1: restore beats a warm cache by >10 s

PROPOSED H2: siblings from one snapshot fail together more than strangers

2009: else: VM_wait(ID)

2026: else: reduce(receipts) $\rightarrow N_{\text{eff}}$, or abstain (proposed)

 **Forks multiply executions, not evidence.**

Help wanted: estimating ρ from lineage; leases that fork.

arXiv:2607.09689